Lab 2: Understanding YAML Files and Pod Creation in Kubernetes

Objective

The objective of this lab is to understand:

- What YAML files are
- Why YAML is used in Kubernetes
- YAML structure and syntax rules
- Creating Pods using YAML
- Deploying and managing Pods using kubectl commands

What is YAML?

YAML (YAML Ain't Markup Language) is a human-readable data serialization language used for configuration files.

It is widely used in:

- Kubernetes
- Docker Compose
- Ansible
- Jenkins Pipelines
- GitHub Actions

YAML uses:

- Indentation
- Key-value pairs
- Lists

which makes configuration files easy to read and maintain.

Why YAML is Used in Kubernetes?

Kubernetes uses YAML files to define the desired state of applications and infrastructure.

Benefits:

- Automation
- Reusability
- Consistency
- Version Control
- Infrastructure as Code (IaC)

Instead of manually creating resources every time, engineers define them once in a YAML file and deploy them repeatedly.

YAML Rules

Rule 1: Key-Value Format

```
name: nginx-pod
```

Rule 2: Parent and Child Relationship

```
metadata:
  name: nginx-pod
```

Child values must be indented under the parent key.

Rule 3: Proper Indentation

```
metadata:
  name: nginx-pod
```

Use spaces only.

Do not use tabs.

Rule 4: Multiple Properties

```
metadata:
  name: nginx-pod
  labels:
    app: nginx
```

Rule 5: Lists Use "-"

```
containers:
- name: nginx
```

Rule 6: List Item Properties

```
containers:
- name: nginx
  image: nginx
```

Rule 7: Multiple List Items

```
containers:
- name: nginx
  image: nginx

- name: busybox
  image: busybox
```

Rule 8: Kubernetes Field Names are Case-Sensitive

Correct:

```
name:
image:
labels:
```

Incorrect:

```
Name:
Image:
Labels:
```

Rule 9: Use Spaces Instead of Tabs

Correct:

```
metadata:
  name: pod1
```

Rule 10: Think in Tree Structure

```
Pod
├── apiVersion
├── kind
├── metadata
│   ├── name
│   └── labels
│       └── app
└── spec
    └── containers
        └── container1
            ├── name
            └── image
```

Basic Structure of a Kubernetes YAML File

Every Kubernetes YAML file generally contains four important sections:

apiVersion

Defines which Kubernetes API version should be used.

Examples:

- v1
- apps/v1
- batch/v1

---

kind

Defines the type of Kubernetes resource.

Examples:

- Pod
- Deployment
- Service
- ConfigMap
- Namespace

---

metadata

Contains information about the resource.

Examples:

- Resource name
- Labels
- Namespace

Example:

```
metadata:
```

```
  name: nginx-pod
```

---

spec

Contains the desired configuration of the resource.

For Pods, it may define:

- Containers
- Images
- Ports
- Volumes
- Environment Variables

Pod Creation Using YAML

Step 1: Create a YAML File

```
vim pod.yaml
```

Step 2: Write Pod Configuration

```
apiVersion: v1
kind: Pod

metadata:
  name: nginx-pod
  labels:
    app: nginx

spec:
  containers:
  - name: nginx-container
    image: nginx
    ports:
    - containerPort: 80
```

Step 3: Create the Pod

```
kubectl apply -f pod.yaml
```

Step 4: Verify Pod Creation

```
kubectl get pods
```

Step 5: Inspect Pod Details

```
kubectl describe pod nginx-pod
```

Step 6: Check Pod Logs

```
kubectl logs nginx-pod
```

Step 7: Access Pod Terminal

```
kubectl exec -it nginx-pod -- /bin/sh
```

Step 8: Delete the Pod

Using file:

```
kubectl delete -f pod.yaml
```

Or using resource name:

```
kubectl delete pod nginx-pod
```

Key Learnings

By completing this lab, we learned:

- YAML fundamentals
- YAML syntax rules
- Kubernetes YAML structure
- Pod creation using YAML
- Single-container Pod configuration
- Multi-container Pod concepts
- Pod deployment using kubectl
- Pod inspection and troubleshooting commands
- Pod deletion

Lab Status: Completed Successfully ✅